Laboratory assignment

Component (3)

Authors: Purdea Bogdan-Florin, Todoran Ciprian Bogdan
Group: ICA2

April 21, 2026

1 Conceptual Modeling

Environment (E)
Represented by the set of possible states. A state is defined as:

- Simulation initialization parameters: Strategies to be played, Payoff chosen values (T, R, P, S), Maximum number of rounds per game
- Current Round Number
- Cumulative Scores

Manager Agent

- **Perception:**
  - The current state of the environment
  - Actions received from player agents
- **Action:**
  - Action requests sent to player agents
  - Round results, including opponent actions and computed payoffs
  - Final match summaries
- **Goal:**
  - Initialize and manage matches between player agents
  - Enforce synchronization across simulation rounds
  - Collect player actions and compute corresponding payoffs
  - Maintain the global state of the simulation
  - Persist results to .CSV storage
- **Environment:** E
- **State:** S.

Player Agents

- **Perception:**
  - Action requests from the Manager Agent

  - Information regarding previous interactions

- **Action:** Selected action: Cooperate or Defect.
- **Goal:**
  - Execute a predefined decision-making strategy
  - Maintain an internal history of interactions
  - Determine actions based on strategy and historical data
- **Environment:** E
- **State:** Internal history of interactions and local knowledge.

2 Properties of the Environment

- **Accessible:** The Manager Agent is the agent that controls and has access to the environment. Since the environment's state changes sequentially, the information provided by the environment is complete, accurate, and up-to-date.
- **Deterministic:** Any action can lead to only one state.
- **Episodic:** There is no past performance evaluation throughout the execution of the agents. The performance of all Player agents is aggregated and compared after all the games are played out.
- **Static:** The environment changes only through the actions of the agents. There are no outside factors influencing it.
- **Discrete:** We limit the number of rounds per game, so all possible actions and percepts are finite.
- **Markovian:** Although some play strategies rely on knowing past moves, we cumulate all previous moves and scores from one state to another. All agents can take actions based only on the information provided by the current state of the environment.

3 Design of the Multi-Agent System

Figure 1 illustrates the class diagram following the PAGE(S) conceptual model for our generalized Prisoner's Dilemma Multi-Agent System. A central **ManagerAgent** instance is tasked with the execution of the game and with the coordination of multiple **PlayerAgent** instances who use specific strategies that implement the **Strategy** interface to define the decision of whether to cooperate or defect. The system relies on an **Environment** to maintain the **State** of the simulation, while **Percept** and **Action** classes facilitate the information flow and decision-making loop between the participants.

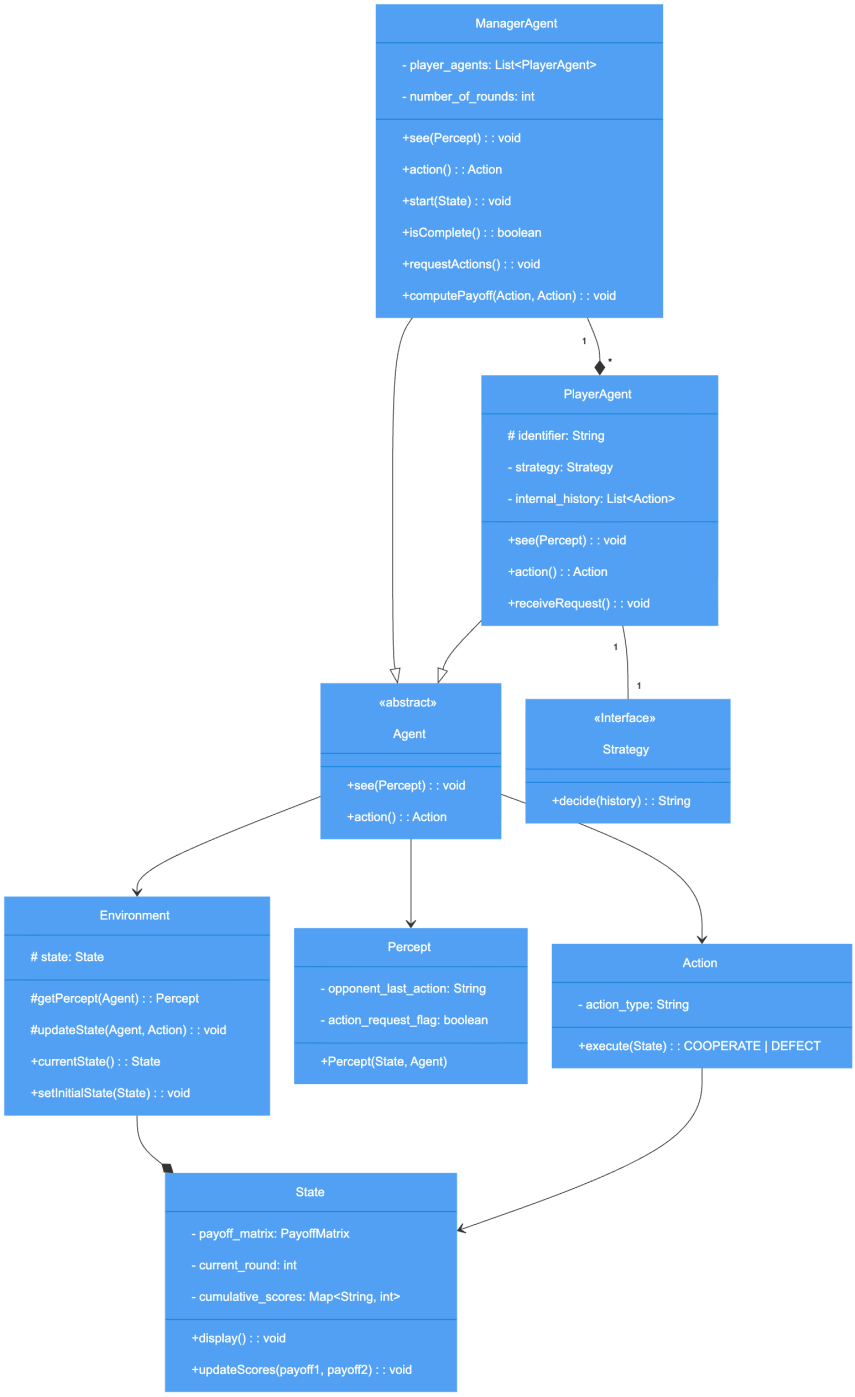

Figure 1: Class Diagram - PAGES modeling

The class diagram in Figure 2 maps the conceptual Multi-Agent System to the specific constraints and capabilities of the SPADE framework. A **ManagerAgent** updated the **Environment** and oversees several **PlayerAgents**, each of which relies on a specific strategies that implement the **Strategy** interface to make decisions that can be based on past actions. Both agent types inherit from SPADE's **Agent** class and define an acting behaviour using predefined classes such as **CyclicBehaviour** from SPADE. Communication between agents is done using the predefined **Message** class provided by SPADE. The **State** manages the logic and scoring of repetitive rounds of play by tracking numerical and historical data.

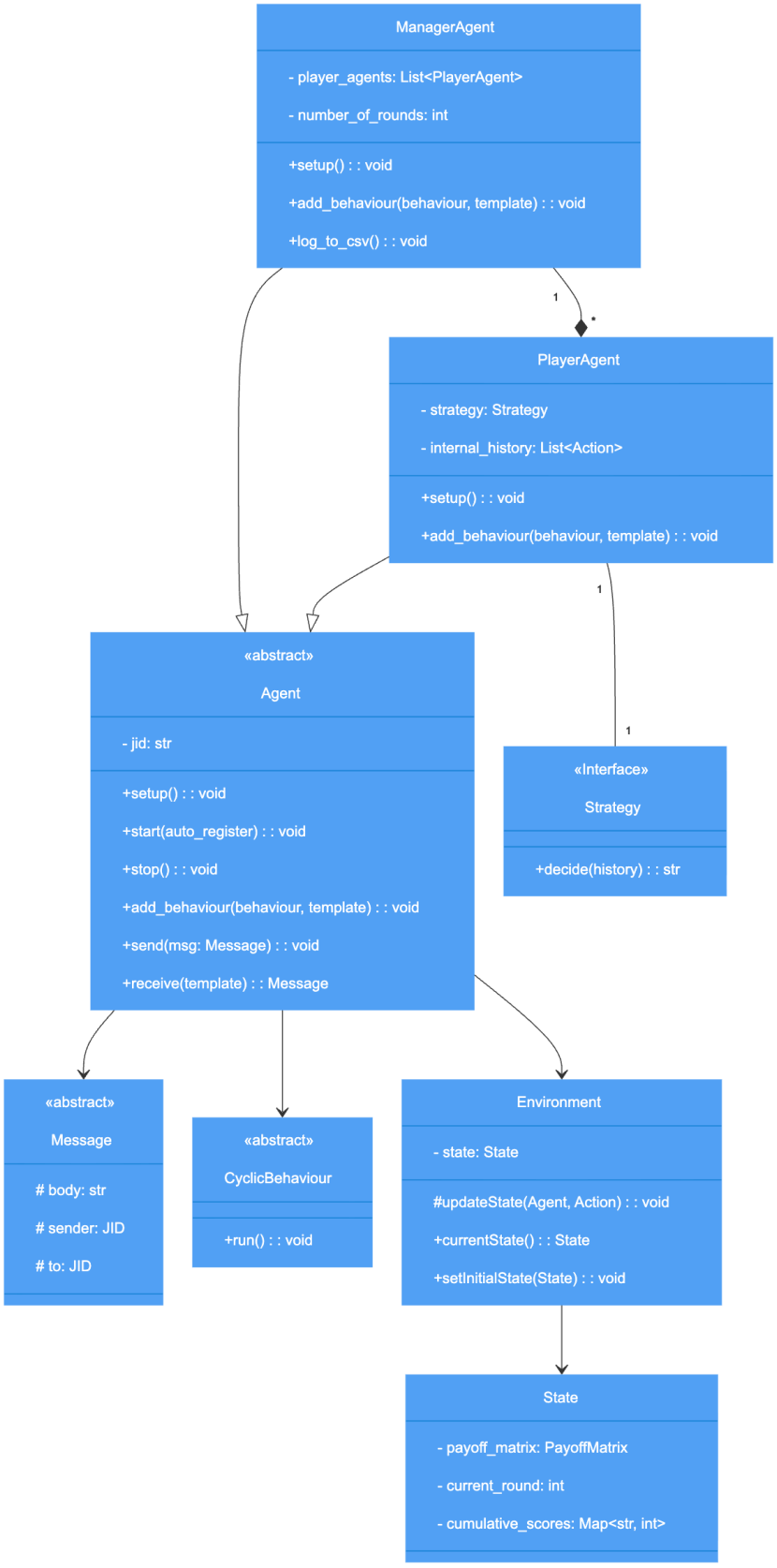

Figure 2: Class Diagram - SPADE implementation

4 Agent Communication and Interaction Model

Communication between agents is implemented using the XMPP-based messaging infrastructure provided by the SPADE framework. The system follows an asynchronous message-passing model.

The agent interaction for a single round proceeds in the following sequential order:

1. **Request:** The Manager Agent sends an action request to each Player Agent at the beginning of a round.
2. **Response:** Each Player Agent responds with its selected action (Cooperate or Defect).
3. **Computation:** The Manager Agent computes the resulting payoffs using the predefined payoff values.
4. **Feedback:** The Manager Agent communicates the round outcome to both Player Agents.

This sequence is repeated for a predefined number of rounds, ensuring proper synchronization and consistency of the simulation.